



Course : Diploma in Electronic & Computer Engineering (EGDF20)

Module : EGE322 – IoT System Project

Assignment 2

In this exercise, you will set up the ESP32 microcontroller with the BME280 environmental sensor and configure a **web server** to display the temperature, humidity, and pressure readings locally on a computer.

You will also configure two sets of ON and OFF buttons on the web page to control two LEDs connected to the ESP32 independently.

The assignment is divided into 4 parts:

Part 1: Set up the BME280 sensor to display: Temperature, Humidity & Pressure

Part 2: Develop a web server using: MicroPython, HTML, CSS & JavaScript

The web server should display the BME280 sensor readings in real time.

Part 3: Create two sets of ON and OFF buttons on the web page to control two LEDs separately.

Part 4: Combine Part 2 and Part 3 into a single web page.

Besides using the default web page design from Parts 2 and 3, you may also improve the web page appearance to make it more attractive and user-friendly.

You will be provided with reference scripts for:

- displaying the BME280 sensor readings through a web server, and
- controlling the LEDs using web-based buttons.

Part 1

Temperature & Humidity Sensing Using BME280

- a) The BME280 sensor module reads barometric pressure, temperature and humidity.
- b) The sensor uses the I2C communication protocol, so the wiring is simple.:

BME280	ESP32
Vin	3,3V
GND	GND
SCL	GPIO 22
SDA	GPIO 21

- c) The BME280 MicroPython library is not included in MicroPython by default. Therefore, you need to upload the library file to the ESP32 and save it as **BME280.py**.
- d) Open a new file and copy the following code into it.

```

main.py
from machine import Pin, I2C
from time import sleep
import BME280
# ESP32 - Pin assignment
i2c = I2C(scl=Pin(22), sda=Pin(21), freq=10000)
while True:
    bme = BME280.BME280(i2c=i2c)
    temp = bme.temperature()
    hum = bme.humidity()
    pres = bme.pressure()
    # uncomment for temperature in Fahrenheit
    #temp = (bme.read_temperature()/100) * (9/5) + 32
    #temp = str(round(temp, 2)) + 'F'
    print('Temperature: ', temp)
    print('Humidity: ', hum)
    print('Pressure: ', pres)
    sleep(5)

```

- e) Save the file as **main.py** and run the program.
- f) You should see the temperature, humidity, and pressure readings displayed every 5 seconds.

Part 2

- a) Next, you need to modify the python code below that will display the bme280 sensor reading from:

BME280 Sensor Readings**Temperature (C):** 20.81C**Pressure (hPa):** 1002.56hPa**Humidity (%):** 47.57%

to

ESP with BME280

MEASUREMENT	VALUE
Temp. Celsius	29.10C
Pressure	1003.56hPa
Humidity	47.66%

using table in html.

- b) The original code that will display the bme280 reading as text:
(reference: *bme280_WebServer_1.py* with *boot_1.py*(*boot.py*))

def web_page():

bme = bme280.BME280(i2c=i2c)

html = """

<html>

<head>

<meta http-equiv="refresh" content="5">

<title>BME280 Sensor Readings</title>

</head>

<body>

<h1>BME280 Sensor Readings</h1>

<p>Temperature (C): {}</p>

<p>Pressure (hPa): {}</p>

<p>Humidity (%): {}</p>

</body>

```
</html>
"".format(bme.temperature, bme.pressure, bme.humidity)
return html
```

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(('', 80))
s.listen(5)
```

```
while True:
    try:
        if gc.mem_free() < 102000:
            gc.collect()
        conn, addr = s.accept()
        conn.settimeout(3.0)
        print('Got a connection from %s' % str(addr))
        request = conn.recv(1024)
        conn.settimeout(None)
        request = str(request)
        print('Content = %s' % request)
        response = web_page()
        conn.send('HTTP/1.1 200 OK\n')
        conn.send('Content-Type: text/html\n')
        conn.send('Connection: close\n\n')
        conn.sendall(response)
        conn.close()
    except OSError as e:
        conn.close()
        print('Connection closed')
```

Part 3

- a) In this exercise, you will need to configure two pairs of software switches to turn on and off two LEDs.

(reference: web_server_leded_on-off.py with boot_2.py(boot.py))

boot.py

```
# LED WEB SERVER

try:
    import usocket as socket
except:
    import socket

from machine import Pin
import network
import esp
import gc

esp.osdebug(None)
gc.collect()

ssid = 'Your_SSID' # Enter your Wifi SSID
password = 'Your_Password' # Enter your password

station = network.WLAN(network.STA_IF)
station.active(True)
station.connect(ssid, password)

while station.isconnected() == False:
    pass
print('Connection successful')
print(station.ifconfig())

led = Pin(2, Pin.OUT) # Change to your LED GPIO Pin
```

main.py

```

def web_page():
    if led.value() == 1:
        gpio_state="ON"
    else:
        gpio_state="OFF"

    html = """<html><head> <title>ESP Web Server</title> <meta
name="viewport" content="width=device-width, initial-scale=1">
    <link rel="icon" href="data:,"> <style>html{font-family: Helvetica;
display:inline-block; margin: 0px auto; text-align: center;}
    h1{color: #0F3376; padding: 2vh;}p{font-size: 1.5rem;}.button{display:
inline-block; background-color: #e7bd3b; border: none;
    border-radius: 4px; color: white; padding: 16px 40px; text-decoration: none;
font-size: 30px; margin: 2px; cursor: pointer;}
    .button2{background-color: #4286f4;}</style></head><body> <h1>ESP
Web Server</h1>
    <p>LED2 state: <strong>"""" + gpio_state + """"</strong></p><p><a
href="/?led=on"><button class="button">ON</button></a></p>
    <p><a href="/?led=off"><button class="button
button2">OFF</button></a></p></body></html>""""
    return html

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(('', 80))
s.listen(5)

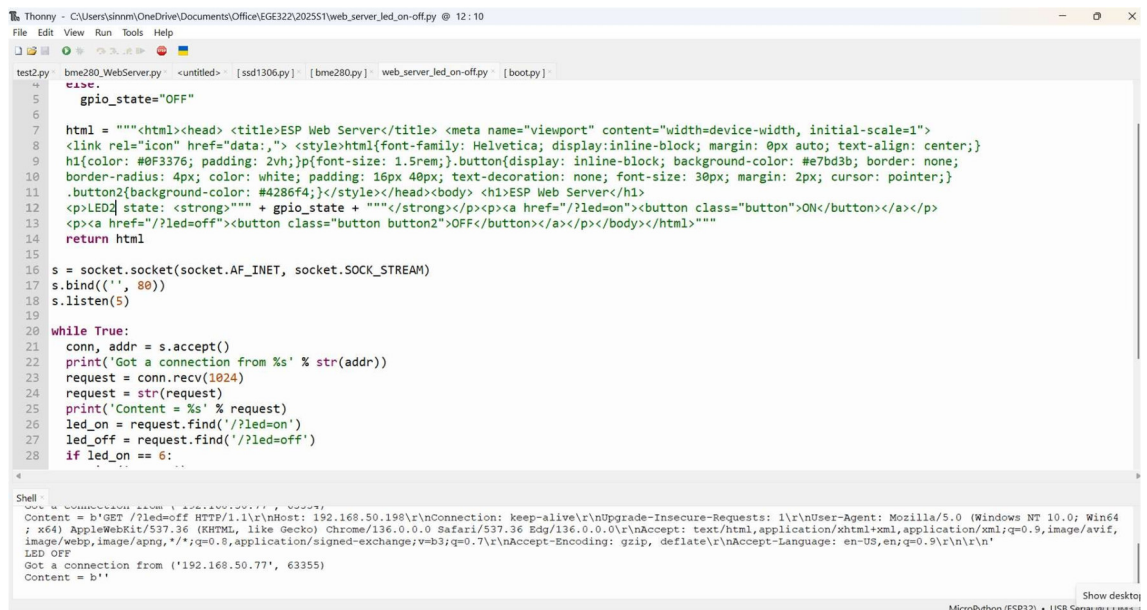
while True:
    conn, addr = s.accept()
    print('Got a connection from %s' % str(addr))
    request = conn.recv(1024)
    request = str(request)
    print('Content = %s' % request)
    led_on = request.find('/?led=on')
    led_off = request.find('/?led=off')

```

```

if led_on == 6:
    print('LED ON')
    led.value(1)
if led_off == 6:
    print('LED OFF')
    led.value(0)
response = web_page()
conn.send('HTTP/1.1 200 OK\n')
conn.send('Content-Type: text/html\n')
conn.send('Connection: close\n\n')
conn.sendall(response)
conn.close()

```



The screenshot shows a Thonny IDE window with a Python script for an ESP Web Server. The script defines a function `web_page()` that returns an HTML response with a button to toggle the LED state. The main loop listens for connections and processes requests to turn the LED on or off.

```

1  #!/usr/bin/env python
2
3  import sys
4  import os
5  import time
6  import random
7  import socket
8  import threading
9  import json
10 import hashlib
11
12 # GPIO pins
13 LED_PIN = 16
14
15 # Web server
16 def web_page():
17     html = """<html><head> <title>ESP Web Server</title> <meta name="viewport" content="width=device-width, initial-scale=1">
18     <link rel="icon" href="data:;"/> <style>html{font-family: Helvetica; display:inline-block; margin: 0px auto; text-align: center;}
19     h1{color: #0F3376; padding: 2vh;}p{font-size: 1.5rem;}.button{display: inline-block; background-color: #e7bd3b; border: none;
20     border-radius: 4px; color: white; padding: 16px 40px; text-decoration: none; font-size: 30px; margin: 2px; cursor: pointer;}
21     .button2{background-color: #4286f4;}/></style></head><body> <h1>ESP Web Server</h1>
22     <p>LED state: <strong>"" + gpio_state + ""</strong></p><p><a href="/?led=on"><button class="button">ON</button></a></p>
23     <p><a href="/?led=off"><button class="button button2">OFF</button></a></p></body></html>"""
24     return html
25
26 # Main loop
27 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
28 s.bind(('', 80))
29 s.listen(5)
30
31 while True:
32     conn, addr = s.accept()
33     print('Got a connection from %s' % str(addr))
34     request = conn.recv(1024)
35     request = str(request)
36     print('Content = %s' % request)
37     led_on = request.find('/?led=on')
38     led_off = request.find('/?led=off')
39     if led_on == 6:
40         # Turn LED on
41         print('LED ON')
42         led.value(1)
43     elif led_off == 6:
44         # Turn LED off
45         print('LED OFF')
46         led.value(0)
47     conn.send('HTTP/1.1 200 OK\n')
48     conn.send('Content-Type: text/html\n')
49     conn.send('Connection: close\n\n')
50     conn.sendall(response)
51     conn.close()

```

The Shell output shows the following:

```

Got a connection from ('192.168.50.198', 63355)
Content = b'GET /?led=off HTTP/1.1\r\nHost: 192.168.50.198\r\nConnection: keep-alive\r\nUpgrade-Insecure-Requests: 1\r\nUser-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36 Edg/136.0.0.0\r\nAccept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7\r\nAccept-Encoding: gzip, deflate\r\nAccept-Language: en-US,en;q=0.9\r\n\r\n'
LED OFF
Got a connection from ('192.168.50.77', 63355)
Content = b''

```

Output: <http://192.168.50.198/?led=off>

ESP Web Server

LED state: **OFF**

ON

OFF

b) Modify the python code to add in another LED such that you have:

ESP Web Server

LED1 state: **OFF**

ON

OFF

LED2 state: **OFF**

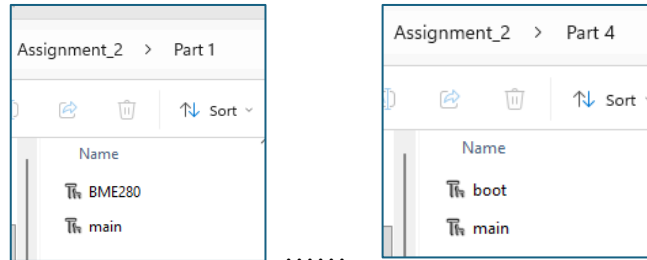
ON

OFF

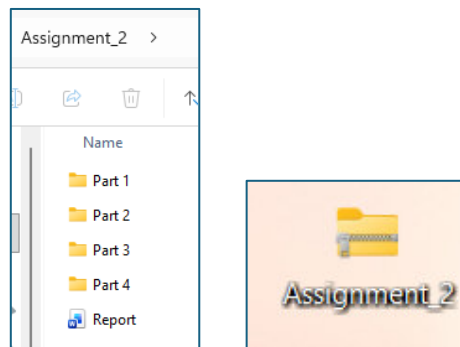
c) You can connect the LED1 and LED2 to any digital GPIO port.

Exercise Submission

- 1) Organize your files into the following folders:
 - **Part 1 Folder: Main File and Library File**
 - **Part 2 Folder: Main File and Boot File**
 - **Part 3 Folder: Main File and Boot File**
 - **Part 4 Folder: Main File and Boot File**



- 2) Create a **Microsoft Word** document containing the following:
 - Screenshot of the Part 2 result.
 - Highlight the modified sections in the Part 2 main.py file compared to the original code provided.
 - Screenshot of the Part 3 result.
 - Highlight the modified sections in the Part 3 main.py file compared to the original code provided.
 - Include the final screenshot of Part 4 showing the combined web page from Part 2 and Part 3.
- 3) After completing all parts, place all folders and the Word document into a folder named **Assignment_2**.



- 4) Compress the folder into a ZIP file and submit it to Brightspace.

---THE END---